

Companion di studio, Modulo 3 (materiale extra per l'orale)

Materiale di studio personale, separato dal report consegnato (che non viene toccato). Copre: le risposte ai dubbi del todo, il walkthrough del report figura per figura, i sei esperimenti aggiuntivi (01..06_*/), i deep dive sui costrutti OpenMP usati nel codice, i punti di onestà e il cheat-sheet finale.

Tutti i numeri citati sono misurati: o vengono dai CSV del report (results/cluster/), o dai CSV degli esperimenti extra (0N_*/results/), girati sullo stesso tipo di nodo (Ivy Bridge, Xeon E5-2640 v2, 2x8 core, 2 thread hardware per core, 2 nodi NUMA, L3 20 MB per socket) con OMP_PROC_BIND=close, OMP_PLACES=cores, NR=10M, NS=20M, P=128, seed 42, max_key=5M, 5 ripetizioni.

1. Risposte rapide ai dubbi del todo

Perché il merge degli istogrammi sta in un `#pragma omp single`? Il merge somma T istogrammi privati (T x P contatori) in uno globale e ne fa il prefix sum: è una riduzione con dipendenza sequenziale sul prefix e un volume di lavoro minuscolo ($TP = 16128 = 2048$ somme, più 128 per il prefix: microsecondi). Parallelizzarlo richiederebbe sincronizzazione aggiuntiva per un lavoro che non paga nemmeno il costo di un fork. `single` (e non `master`) perché non importa quale thread lo faccia, e perché `single` ha una barriera implicita in uscita che gli altri thread devono comunque attraversare prima dello scatter: la dipendenza è strutturale (lo scatter usa gli offset prodotti dal merge).

Perché lo scatter usa `schedule(static)`? Due ragioni. (1) Il lavoro per tupla è costante (hash + una scrittura), quindi non c'è imbalance da assorbire e `static` è la politica a overhead zero. (2) È un requisito di correttezza del trucco dei cursori: `static` garantisce che il thread `t` visiti nello scatter esattamente lo stesso range di input che ha contato nell'istogramma, quindi `cursors[t][pid]` (derivato da `local_hists[t]`) è l'offset esatto di partenza del thread `t` dentro la partizione `pid`, e le scritture non collidono mai senza lock né atomiche. Con una politica dinamica il range visitato cambierebbe e gli offset per-thread non sarebbero più validi.

Da dove viene il rapporto ~290x fra partizione hot e cold? $f_{\text{hot}} = \rho/H + (1-\rho)/P = 0.9/4 + 0.1/128 = 0.2258$ (22.6% dei record su una hot); $f_{\text{cold}} = (1-\rho)/P = 0.1/128 = 0.00078$ (0.078% su una cold). Rapporto $0.2258/0.00078 \approx 289$. Il primo termine di f_{hot} è la quota del pool hot che cade su quella chiave (le 4 chiavi hot stanno su 4 partizioni distinte per costruzione del generatore), il secondo è la quota uniforme che ci finisce comunque sopra.

dynamic, 1 sul join: quanto costa davvero il dispatch? Esp. 1: a parità di tutto, la differenza mediana con `static` sul join resta sotto 1.3 ms a $P=128$ e sotto 0.7 ms fino a $P=8192$, dove le iterazioni sono 32 volte più piccole. Il dispatch è una fetch-and-add sul contatore condiviso del loop: decine di nanosecondi per iterazione, irrilevante quando un'iterazione vale centinaia di microsecondi.

Il `nowait` sul `single` del join task è davvero necessario? No, ed è un punto da ammettere con onestà (esp. 2): con e senza `nowait` i tempi coincidono entro il rumore. La barriera implicita del `single` è un *task scheduling point*: i thread fermi alla barriera eseguono i task pendenti invece di aspettare. Il `nowait` è innocuo (evita al runtime il passaggio dalla barriera) ma la frase del report “senza sarebbero fermi” è tecnicamente imprecisa.

Tied vs untied. I task sono tied (default): un task sospeso riprende solo sul thread che l'aveva iniziato. Nel codice ogni body legge `omp_get_thread_num()` e lo usa per indicizzare `thr_results[tid]`: fra la lettura e l'uso non c'è nessun task scheduling point, quindi l'indice è stabile. Con task untied un task potrebbe migrare a un altro thread in un punto di scheduling e `tid` diventerebbe stantio: qui non succederebbe comunque (nessun punto di sospensione nel body), ma untied non darebbe alcun vantaggio, perché i task non si sospendono mai e la granularità è grossa.

Taskgroup vs taskwait vs barriera. `taskgroup` aspetta *tutti i discendenti* generati nel blocco (anche i task annidati), `taskwait` solo i figli diretti. Nel codice: i due `taskgroup` di histogram e scatter servono da barriera di fase (il merge non può partire finché ogni contatore è arrivato); il `taskgroup` annidato dentro il task di una partizione hot garantisce che la tabella condivisa `tbl` (stack del task esterno) sopravviva a tutti i sub-task di probe che la leggono. Qui `taskwait` sarebbe equivalente (i probe sono figli diretti), ma `taskgroup` esprime l'intenzione “aspetta il lavoro che ho generato, incluso ciò che i figli potrebbero generare”. La variante loop non usa niente di tutto ciò: si affida alle barriere implicite dei costrutti worksharing.

LPT e bound di Graham. Longest Processing Time first: si ordinano i job per peso decrescente e ogni job va al worker più scarico (qui: entra prima nella coda). Graham (1969): il makespan di LPT è al più $4/3 - 1/(3m)$ volte l'ottimo su m macchine identiche. Il proxy di costo $|Rp| + |Sp|$ è giusto perché build e probe sono lineari nei rispettivi input. Ma la misura (esp. 3) ridimensiona LPT su questo problema: vale il 2-5% sul join, mentre lo split intra-partizione vale il 15%; con $P/T = 8$ la coda ha abbastanza task piccoli da riempire i buchi comunque.

Perché il build della tabella hot resta sequenziale? `FlatCountMap::increment` fa read-modify-write non atomico sugli slot (probing lineare + contatore): due thread potrebbero scegliere lo stesso slot per chiavi diverse o perdere incrementi. Servirebbero CAS per l'inserimento e fetch-and-add per i contatori, che costano più del guadagno: sotto questo skew il build della partizione hot conta 4 chiavi distinte (tabella minuscola, tutta in L1), è il probe dei 4.5M record a dominare, ed è read-only, quindi si parallelizza gratis. Lo split parte infatti solo sul probe.

Perché la validazione skewed usa il parallelo a $T=1$ come riferimento? Il binario sequenziale consegnato non ha il generatore skewed (arrivato con l'M3); il parallelo a $T=1$ esegue lo stesso identico algoritmo senza concorrenza, quindi è un riferimento valido per il confronto della tripla (join_count, ck1, ck2) a parità di input. Il check aggiuntivo `loop == task` a $T=4$ verifica che le due varianti calcolino la stessa cosa anche in presenza di concorrenza.

Il calcolo dei checksum non andrebbe fuori dal tempo misurato, visto che è "verifica"? No: va distinta la verifica dall'output. I due checksum calcolati nel probe sono l'output della query: il risultato del join è per definizione la tripla (join_count, ck1, ck2), un'aggregazione sui match (come la SUM di una query di aggregazione), non la materializzazione delle coppie. Toglierli dal timing significherebbe o materializzare i match per calcolarli dopo (più lento, cambia il kernel) o non produrre nulla per match, e allora il compilatore potrebbe eliminare il probe come dead code. Il costo (2 splitmix64 per match) è identico in tutte le varianti e in tutti i moduli, quindi non distorce alcun confronto. La VERIFICA vera (naive join $O(N^2)$, confronto col sequenziale) è invece davvero fuori dalla regione misurata.

Perché i checksum sono invarianti all'ordine? Ogni match contribuisce `splitmix64(k)*m` sommato modulo 2^{64} (overflow naturale degli unsigned): la somma è commutativa e associativa, quindi il totale non dipende dall'ordine in cui thread e task completano. `splitmix64` serve a rendere il contributo sensibile alla chiave (un semplice `+m` non distinguerebbe errori compensati); il secondo checksum usa la chiave xorata con la costante aurea per avere una seconda proiezione indipendente.

2. Walkthrough del report, figura per figura

Tab. 1 / Fig. 1 (strong scaling). Quattro curve: {loop, task} x {uniforme, skewed}. Uniforme: loop sempre davanti (a $T=16$, 0.070 contro 0.092 s, gap 31%), e il gap sta tutto in histogram+scatter (esp. 2); il join è identico. Il loop tocca 7.9x relativo a se stesso a $T=16$ (50% di efficienza: kernel memory-bound, la banda satura prima dei core), crolla a $T=20$ (il team a cavallo fra core fisici e SMT: 4 thread condividono 2 core mentre 12 core hanno un thread; le barriere aspettano i 4 lenti) e recupera parzialmente a $T=32$ (SMT simmetrico). Skewed: si inverte, il task vince dal $T=4$ in poi (LPT + split; il grosso è lo split, esp. 3).

Fig. 2 (skew gap). Pannello (a): il tempo del join loop vs task sotto skew; il gap è la firma dello split intra-partizione. Pannello (b): il rapporto $T_{\text{loop}}/T_{\text{task}}$ tocca 1.17 a $T=8$, si restringe a 16-20, riallarga a 1.28 a $T=32$: il loop è inchiodato al collo delle 4 partizioni (tetto $1/f_{\text{hot}} \approx 4.4$ sul join, esp. 3), il task lo aggira.

Fig. 3 (weak scaling). Efficienza = $T(1)/T(p)$ con input proporzionale ai thread. Uniforme: ~80% (loop) dentro il socket, 63% a $T=16$, 26% a $T=32$: il working set dello scatter cresce linearmente con T e la banda aggregata dei due socket satura prima dei core. Il task assorbe in più il costo di creazione task per fase. Skewed peggio: crescere NR fa crescere anche le partizioni hot in proporzione, e il build hot resta sequenziale.

Fig. 4 (breakdown). Nota preliminare su due scelte della figura: (a) " $T=1$ omissso perché domina visivamente" significa solo che la barra sequenziale (~550 ms) è 5-8 volte quelle parallele e su asse condiviso le schiaccerebbe; (b) il taglio a $T=20$ è una scelta editoriale, non di dati: il CSV dello strong scaling ha le fasi a ogni T , e l'esp. 7 (07_breakdown_full/) ricostruisce il breakdown intero, $T=1$ e $T=32$ compresi.

Contenuto della fig. 4. Uniforme: il join domina nel loop (67% a $T=8$); nel task histogram+scatter si gonfiano (+17 ms a $T=8$) e comprimono la quota del join. Skewed: l'imbalance sta solo nel join (histogram e scatter hanno costo per-tupla indipendente dalla distribuzione); a $T=16$ il join è il 76% del totale nel loop, 70% nel task.

Fig. 5 (schedule). A T=8: su uniforme tutte e cinque le politiche entro l'1% (P/T=16 partizioni a thread, lavoro identico comunque distribuito); su skewed le due dynamic a ~80 ms contro ~130 di static/guided (-38%): static incolla le 4 hot a chi capita, dynamic le fa rubare. Esteso in esp. 1 a T={4,16,32} e 7 politiche: la conclusione regge, con la nota che chunk grandi (dynamic,16 e guided,16) degradano a T alto perché 128/16=8 blocchi non bastano per 16-32 thread.

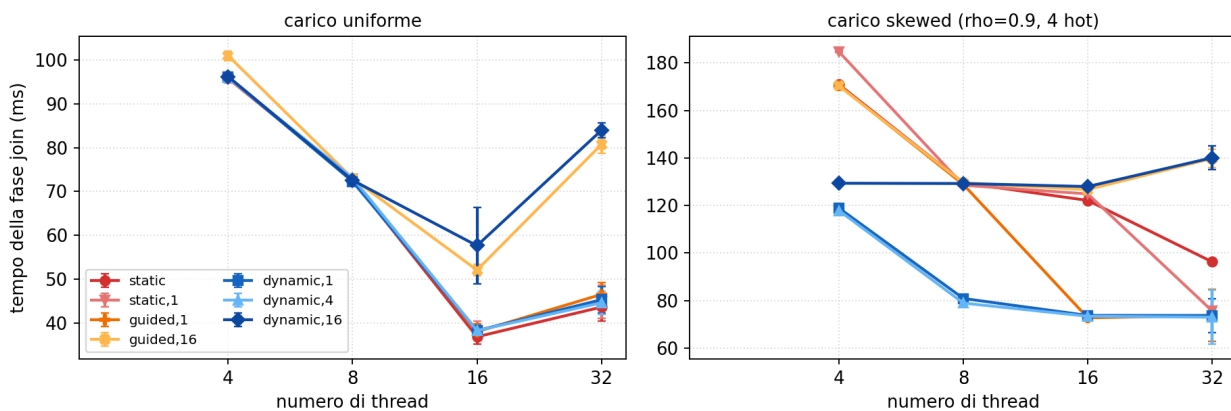
Fig. 6 / Tab. 2 (M2 vs M3). Stesso algoritmo, stesso nodo, stessa baseline (0.802 s): il confronto isola il modello di sincronizzazione più le ottimizzazioni di sez. 2.4. Loop 11.46x contro 7.86x di M2 a T=16. I due contributi grossi: scatter (29 contro 85 ms a T=8, ed è il prefetch: esp. 4 misura 2.1x proprio sullo scatter) e histogram (9 contro 35 ms a T=8). A T=32 convergono (0.086 vs 0.089 s): al tetto di banda la primitiva di sincronizzazione è un dettaglio di secondo ordine.

Efficienza sopra 1 a T basso (eff ≈ 1.22 a T=4). Non è superlinearità: la baseline non ha i prefetch. Esp. 6: con la baseline riottimizzata l'efficienza diventa 1.12, 1.07, 0.95 a T=1,2,4. Vedi sez. 5 (onestà) per come formularlo.

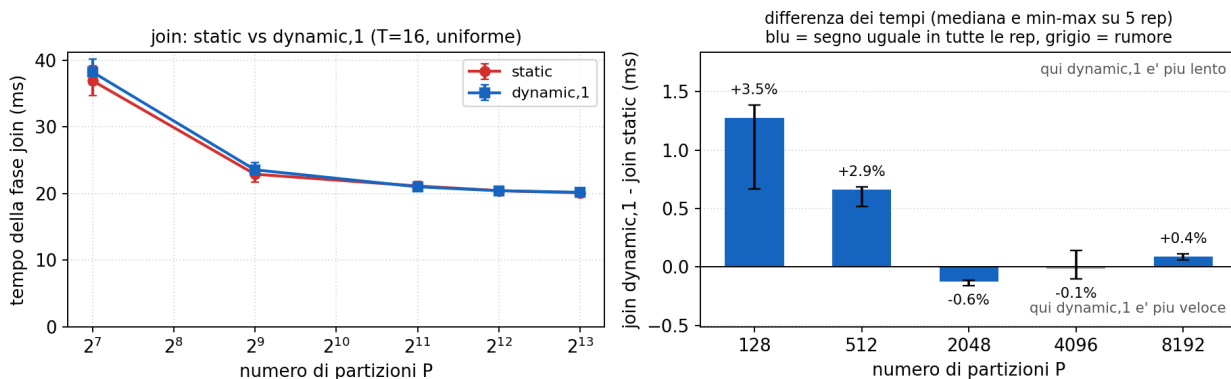
3. I sei esperimenti (spiegazione, meccanismo, conferma)

Esp. 1: schedule del join su tutto il range (01_schedule_sweep/)

Schedule del join su tutto il range di thread (P=128)



Granularità del join: P alto = iterazioni piccole (T=16, uniforme)

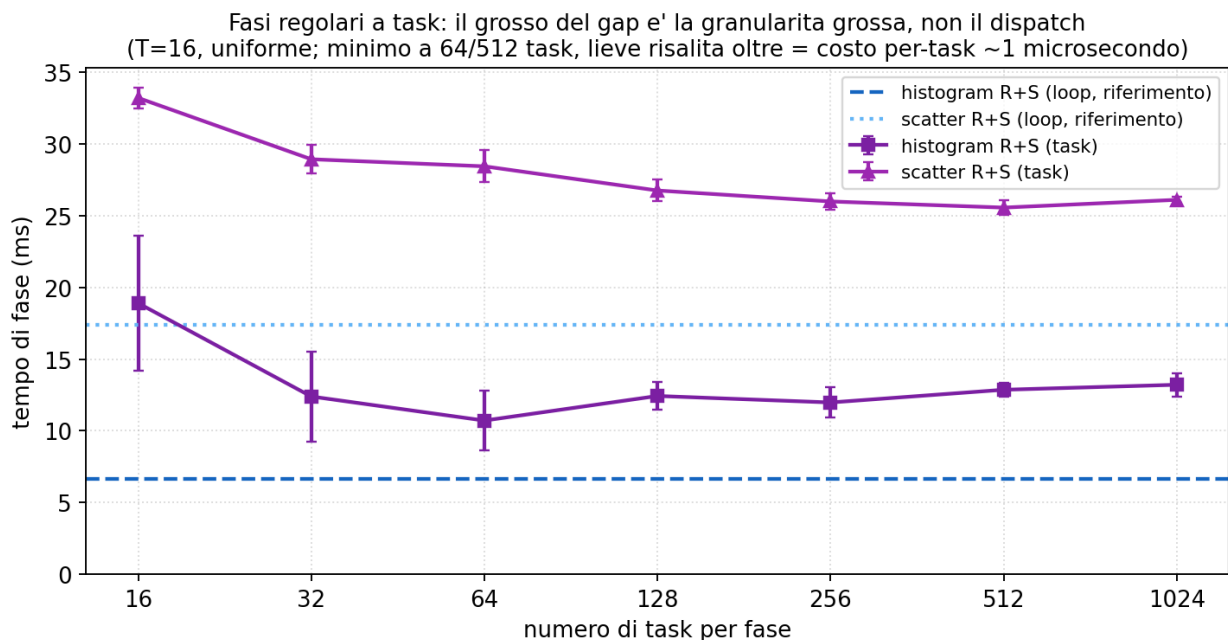


Il report misura la sensibilità solo a T=8. Due domande restavano aperte: la conclusione regge a ogni T? e quanto costa il dispatch dinamico quando le iterazioni sono piccole?

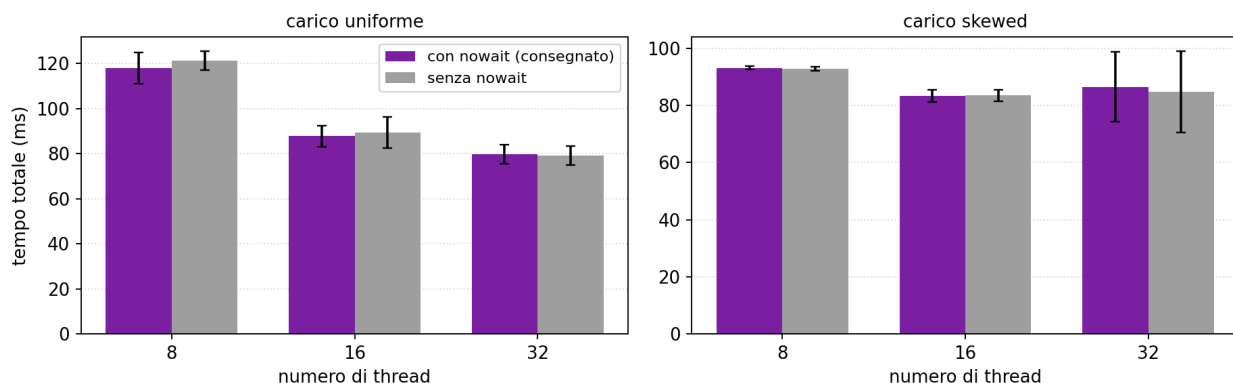
Il binario degli esperimenti (common/omp_ablation.cpp) usa `schedule(runtime) + omp_set_schedule()`, così la politica è un parametro e non una ricompilazione (omp_ablation.cpp:296). Risultati:

- Uniforme: tutte le politiche a chunk piccolo entro il 2% a ogni T. dynamic, 16 e guided, 16 degradano a T=16/32 (58 e 84 ms contro 38 e 45): con chunk 16 su P=128 ci sono solo 8 unità di lavoro, meno dei thread. Lezione: su un loop corto il chunk grande non riduce l'overhead, riduce il parallelismo.
- Skewed: dynamic, 1 e dynamic, 4 migliori a ogni T (74 contro 122-128 ms a T=16). static, 1 recupera a T=32: l'interleaving ciclico pid mod T con T=32 mette le 4 hot su 4 thread diversi per costruzione; a T=8/16 possono collidere sullo stesso thread.
- Granularità (pannello destro della seconda figura, T=16 uniforme, solo fase join): la quantità in y è la differenza dei tempi, join con dynamic, 1 meno join con static, sulla stessa ripetizione. +1.3 ms a P=128 (3.5% della fase, tutte le rep positive), +0.7 ms a P=512; da P=2048 in su la differenza è sotto il punto percentuale e il segno oscilla fra le ripetizioni (barre anche sotto zero: in quei run dynamic è uscito davanti per rumore). Il dispatch è trascurabile a granularità di partizione.
- Bonus: il join scende da 38 a 20 ms alzando P da 128 a 2048 (tabelle per-partizione da 2 MB a ~128 KB: rientrano in cache). Il totale ha il minimo da P=512 in su (48 contro 62 ms): coerente con la partition sensitivity del Modulo 2 (sweet spot 512-1024). P=128 nel report è ereditato per confrontabilità, non è l'ottimo assoluto.

Esp. 2: il costo del modello a task, e il nowait (02_task_overhead/)



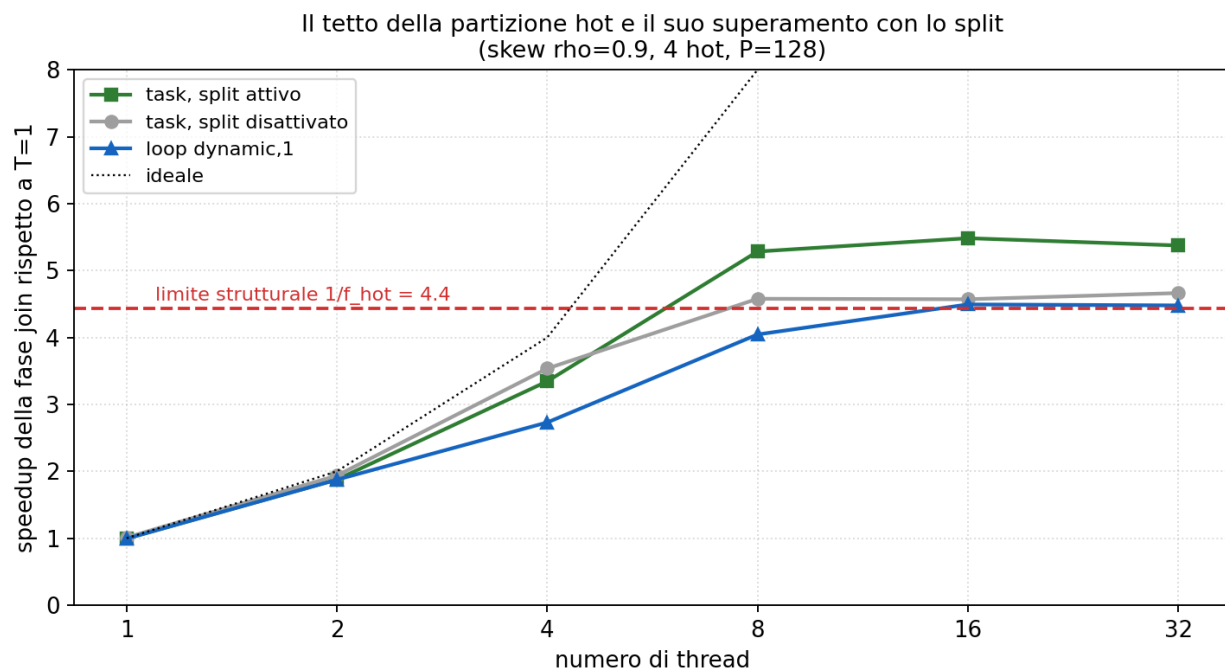
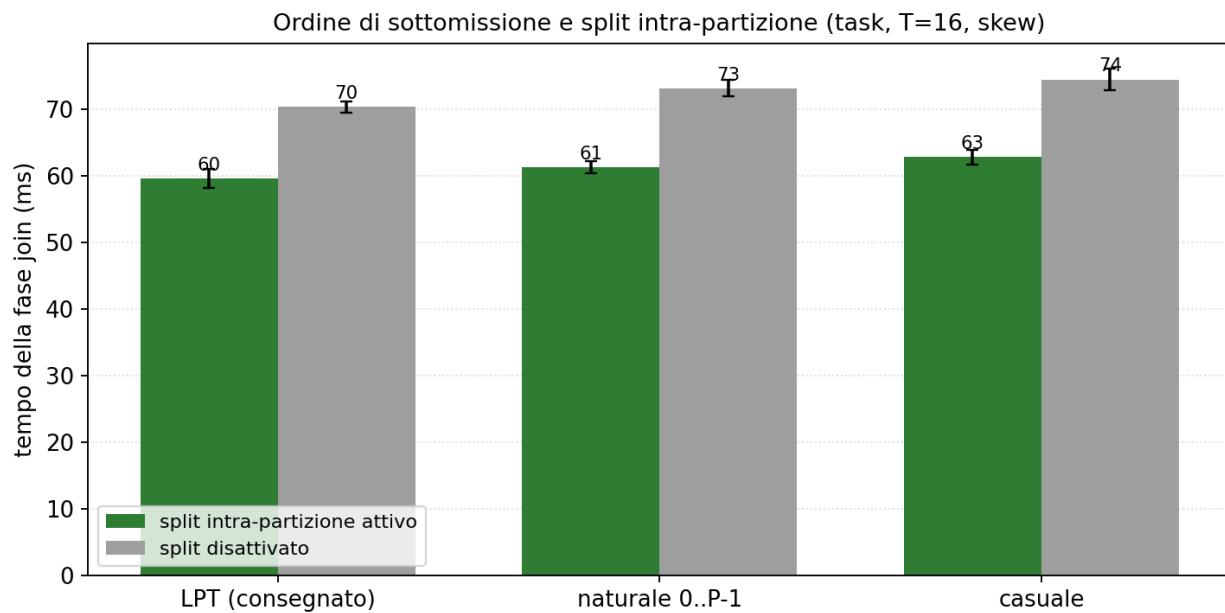
Variante task con e senza nowait sul single: differenza dentro il rumore

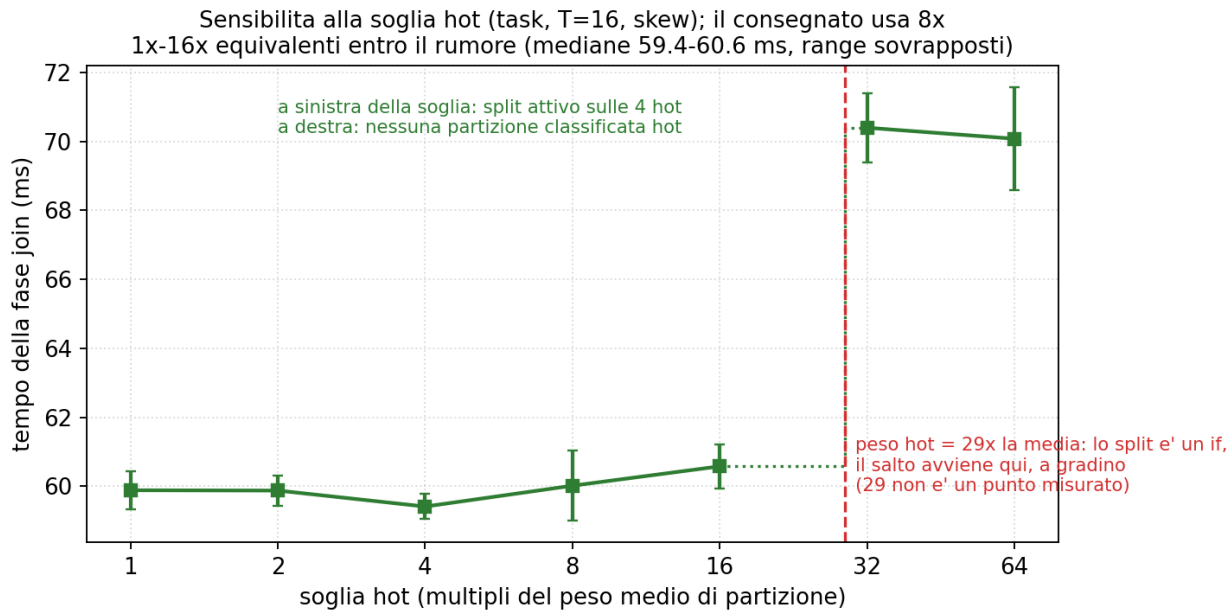


Il gap loop-task su uniforme (31% a T=16) sta tutto nelle fasi regolari. Cosa lo produce? Sweep del numero di task per fase (-tchunks), a T=16 fisso:

- 16 task (consegnato): histogram 19 ms, scatter 33 ms (loop: 6.6 e 17.4).
- Il gap si RIDUCE aumentando i task: histogram ha il minimo a 64 task (10.7 ms, la configurazione più rumorosa: min-max 10.4-15.4), scatter a 512 (25.6). Quindi il grosso del gap a 16 task non è il costo per-task (sarebbe cresciuto col numero), è la granularità grossa: con un task per thread, chi parte in ritardo finisce in ritardo e l'ultimo task chiude la fase; con task più fini la coda riassorbe il jitter di partenza.
- Oltre il minimo le curve risalgono un poco: histogram 13.2 ms a 1024, scatter 26.1 (e qui la risalita è reale: range di 512 e 1024 disgiunti, 24.6-25.9 contro 26.1-26.5). È il costo per-task che diventa visibile: ~2.5 ms per ~1900 task in più su histogram e ~0.5 ms per ~1000 su scatter, cioè circa 1 μ s per task. All'orale: la forma è una U poco profonda, granularità grossa da un lato e overhead per-task dall'altro, con il secondo effetto 5-10 volte più piccolo del primo a queste scale.
- Anche nel punto migliore resta un divario strutturale rispetto al loop (+4 ms histogram, +8 scatter): emissione seriale nel single, gestione della coda, cache locality dei chunk non più allineata ai thread.
- nowait on/off (binario ricompilato con `-DNO_NOWAIT`): 88 contro 89 ms totali a T=16 uniforme, std 5-7 ms. Nessuna differenza misurabile, vedi sez. 1 per la ragione (barriera = task scheduling point).

Esp. 3: LPT vs split intra-partizione (03_lpt_split/)





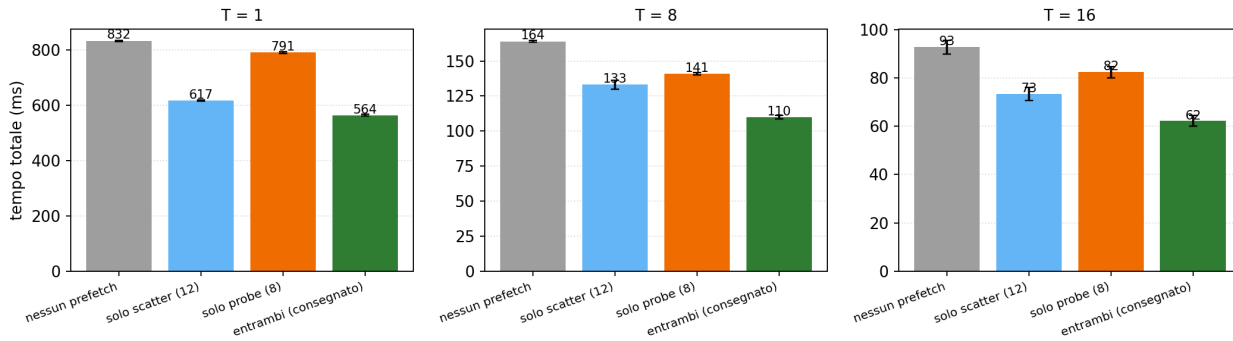
La variante task sotto skew combina LPT e split: chi dei due paga? Matrice {lpt, naturale, casuale} x {split on, off} a T=16 skewed, sul join:

	split on	split off
LPT	59.6 ms	70.4 ms
naturale	61.3 ms	73.1 ms
casuale	62.8 ms	74.4 ms

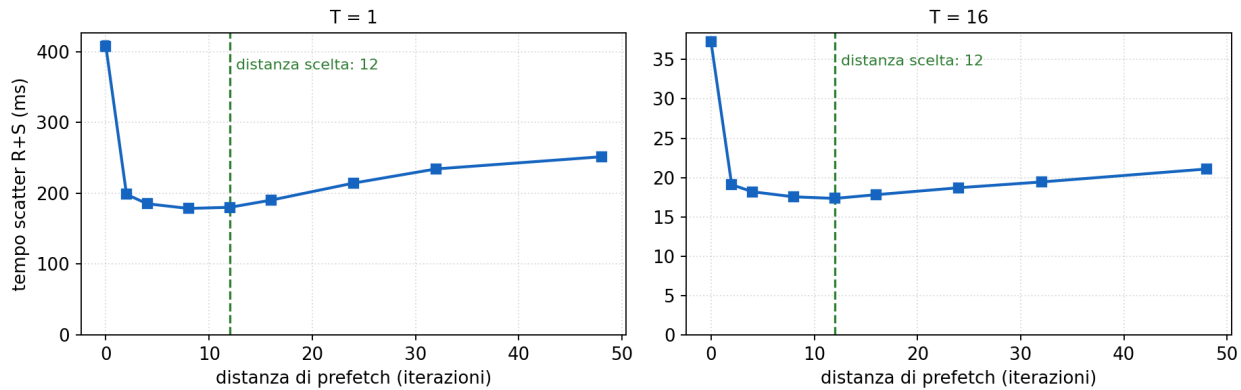
- Lo split vale ~15% qualunque sia l'ordine; LPT vale 2-5% rispetto agli altri ordini. Il claim onesto: il vantaggio task sotto skew è per 3/4 lo split, per 1/4 l'ordine.
- Tetto strutturale: senza split lo speedup del join satura a 4.6 da T=8 (curve task e loop insieme), contro il limite teorico $1/f_{\text{hot}} = 4.4$. Il tetto misurato è leggermente sopra il teorico perché f_{hot} pesa i record, ma il probe della partizione hot costa meno per record della media (tabella di 4 chiavi in L1). Con lo split la curva supera il tetto e arriva a 5.5.
- Soglia hot: piatta da 1x a 16x il peso medio (mediane 59.4-60.6 ms, range sovrapposti: il minimo apparente a 4x non è distinguibile da 8x), poi 70 ms a 32x e 64x. Il peso della hot è $f_{\text{hot}} \cdot P = 29x$ la media e lo split è un if su quel peso: la transizione è un gradino esatto a 29x, non una salita da 16 a 32 (29 non è un punto misurato, è calcolato dai pesi di input). La scelta di 8x è insensibile per costruzione, non un numero magico.

Esp. 4: prefetch software (04_prefetch/)

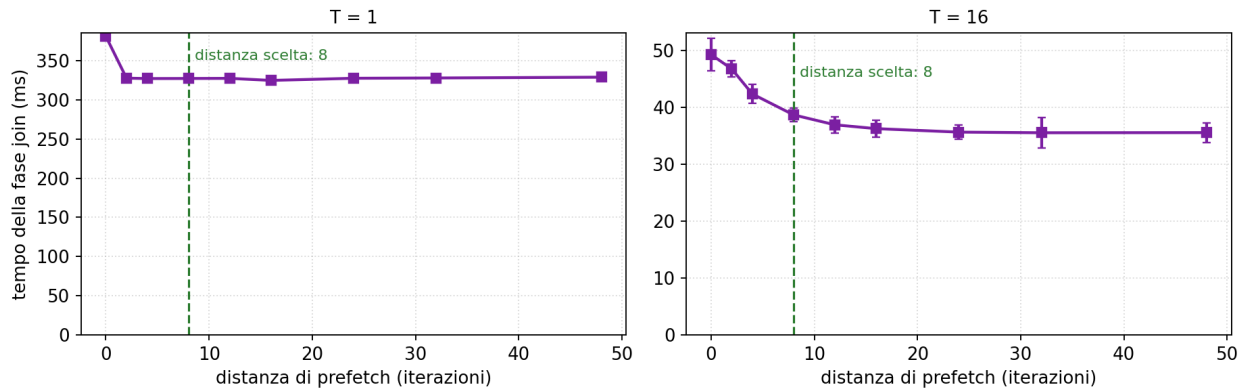
Contributo dei due prefetch al tempo totale (uniforme, P=128)



Distanza di prefetch nello scatter (0 = disattivato; uniforme)



Distanza di prefetch nel probe (0 = disattivato; uniforme)



I due `__builtin_prefetch` del codice: scatter a distanza 12 sullo slot di destinazione (`part.data[cursors[tid][pid_next]` scrittura, hint di non-località temporale), probe a distanza 8 sullo slot della tabella (`tbl.slots[slot_of(k_next)]`, lettura). Entrambe le strutture sono accedute in ordine casuale rispetto allo stream di input: il prefetcher hardware copre lo stream sequenziale (l'input) ma non può indovinare una destinazione che dipende dall'hash della chiave. Il prefetch software calcola l'indirizzo con 12 (o 8) iterazioni di anticipo, cioè paga l'hash due volte per mascherare un miss da centinaia di cicli.

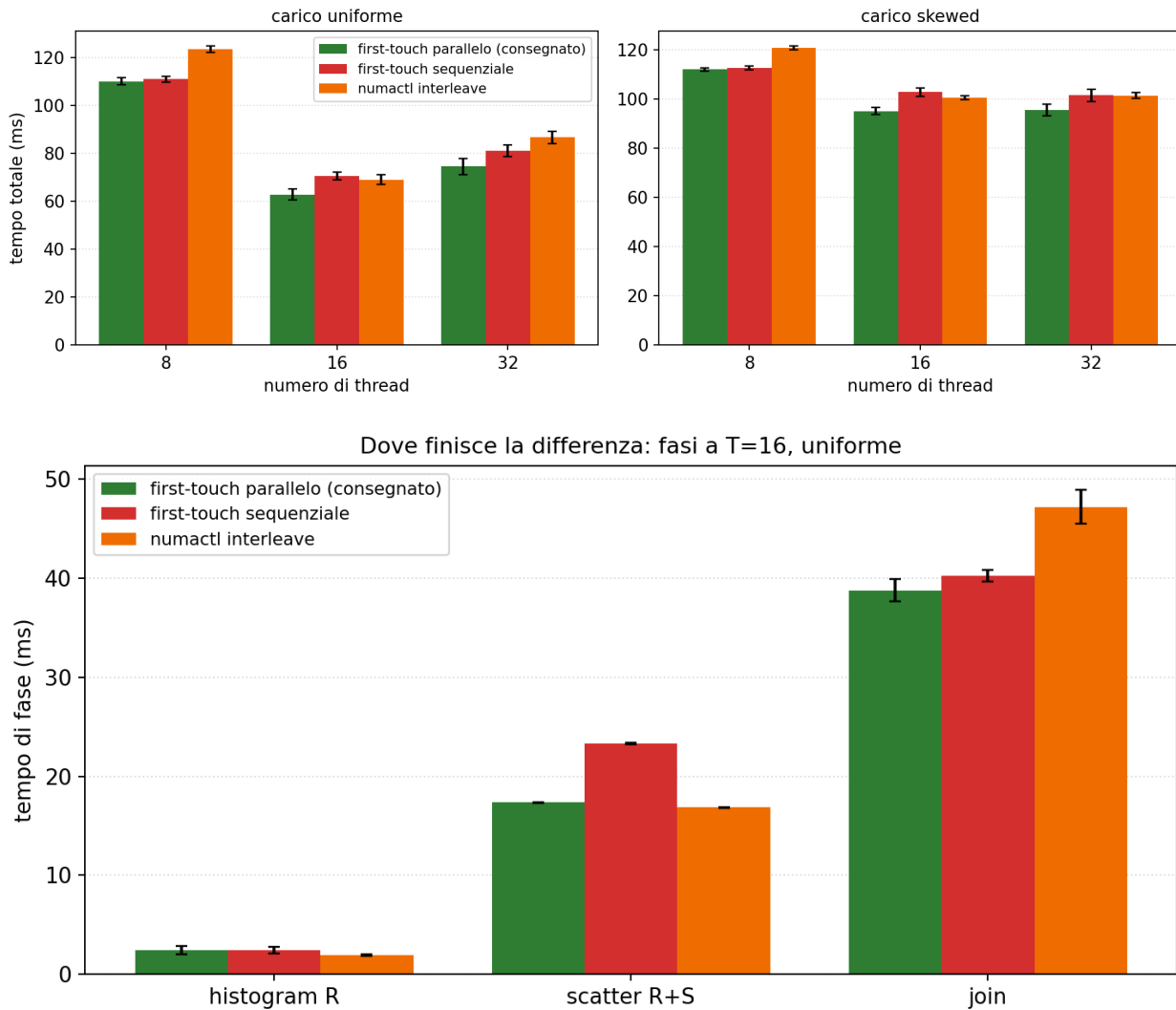
- Ablation 2x2 a T=16: nessuno 93 ms, solo scatter 73, solo probe 82, entrambi 62 ms (1.49x). I contributi si compongono perché mascherano miss indipendenti.
- Scatter: fattore 2.1 sulla fase a ogni T (37 contro 17 ms a T=16). A T=8 misuro 59 contro 29 ms: il conto del

report sul gap M2-M3 (~56 ms attribuiti al prefetch) torna.

- Distanza scatter: curva a U con plateau 8-16; a 2-4 il prefetch arriva tardi, a 48 la riga viene sfrattata o il cursore è già avanzato (+20-40%). 12 sta al centro.
- Distanza probe: a T=1 piatta da 2 a 48 (327-329 ms); a T=16 il minimo scivola a 24-48 (35.6 contro 38.7 ms, -8% sul join, -5% sul totale): con la banda contesa la latenza effettiva cresce e serve più anticipo. Da ammettere senza giri: non serviva alcun tuning per-T, una costante unica 16-24 dominava la scelta consegnata (uguale a T=1, meglio a T=16) a parità di complessità. 8 è stato calibrato sul regime a basso parallelismo; il costo dell'errore è ~3 ms.

Esp. 5: NUMA first-touch (05_numa_firsttouch/)

Piazzamento NUMA dei buffer di partizione (loop, P=128)



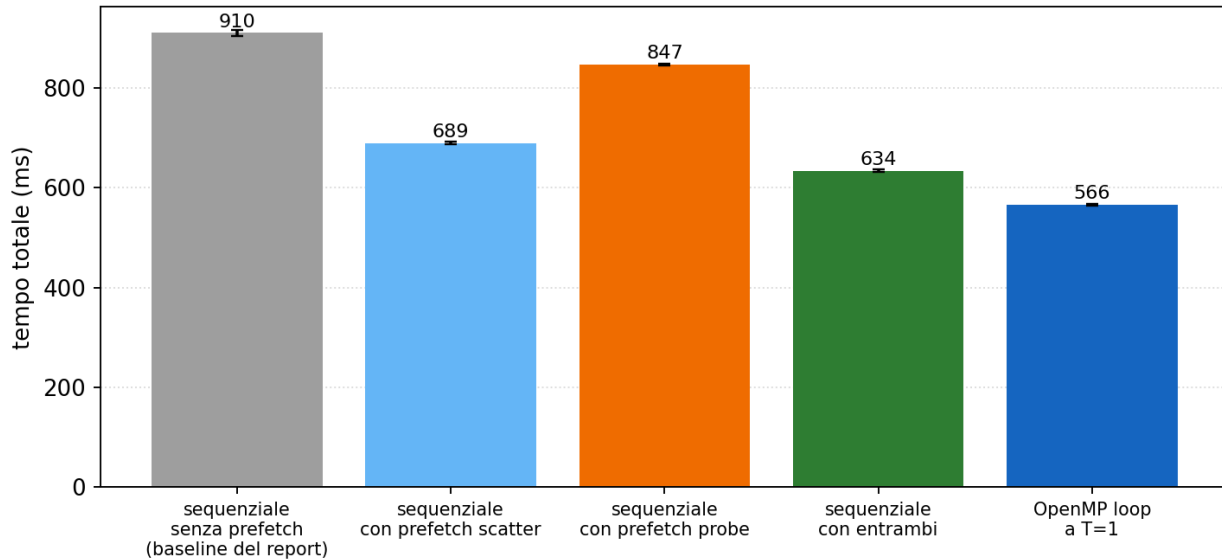
Il meccanismo nel codice consegnato: `PartitionedRelation::data` usa un allocatore che default-inizializza, quindi `resize()` riserva le pagine senza toccarle; lo `zero-init in parallel for schedule(static)` è la prima scrittura di ogni pagina, e sotto Linux first-touch la pagina viene allocata sul nodo NUMA del thread che la tocca. Poiché lo scatter usa la stessa `schedule(static)`, ogni thread riscrive le pagine che ha toccato.

- A T=8 nessuna differenza par/seq: con `OMP_PROC_BIND=c` lose tutto il team sta sul socket 0, dove anche il first-touch sequenziale alloca.
- A T=16 uniforme: par 63, seq 71 ms (+13% sul totale); la differenza è quasi tutta nello scatter (17 contro 23 ms, +35%: metà delle scritture attraversa QPI).

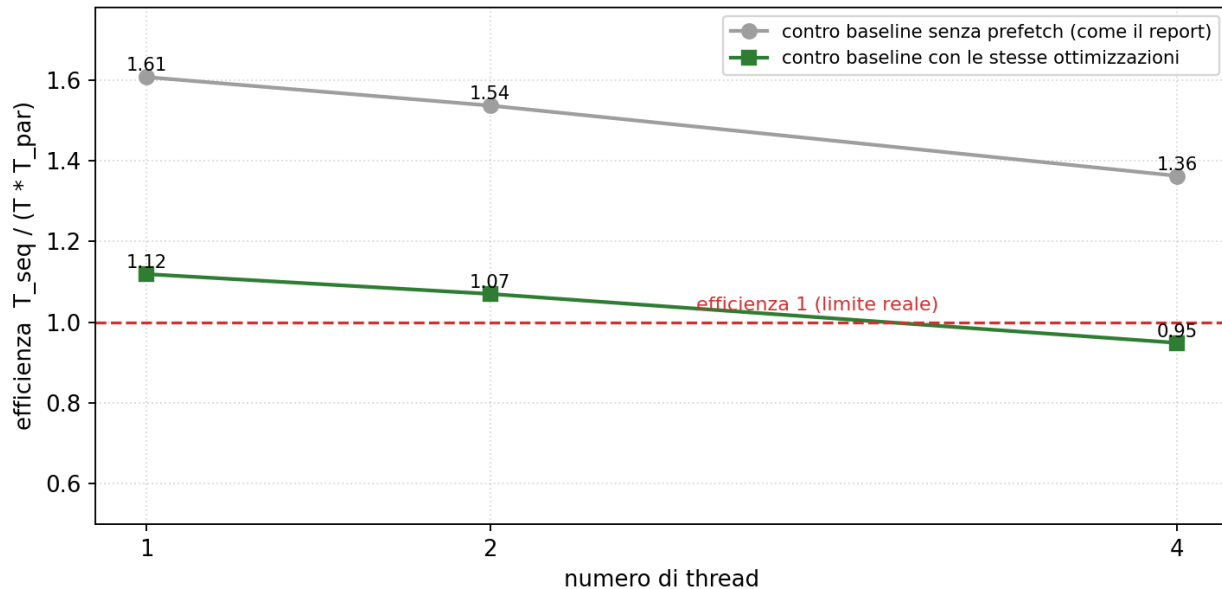
- Interleave (numactl -interleave=all): 69 ms, e colpisce il join (47 contro 39 ms): le letture di build+probe trovano metà dei buffer sul nodo remoto.
- Skewed: stesso pattern, attenuato (le hot sono cache-resident e la quota di traffico remoto pesa meno).

Esp. 6: la superlinearità apparente (06_seq_opts/)

Il rapporto 1.44x a T=1 viene quasi tutto dal prefetch assente nella baseline (uniforme, NR=10M, P=128)



Efficienza a T basso: con la baseline ottimizzata la superlinearità sparisce



common/seq_opts.cpp è la baseline sequenziale con gli stessi prefetch del binario parallelo, attivabili a runtime. Misure (uniforme):

- seq senza prefetch (config. del report): 910 ms. Con solo scatter: 689. Con solo probe: 847. Con entrambi: 634 ms. OpenMP loop a T=1: 566 ms.
- Efficienza a T={1,2,4} contro la baseline nuda: 1.61, 1.54, 1.36 (sopra 1, come nel report). Contro la baseline ottimizzata: 1.12, 1.07, 0.95. La superlinearità sparisce.
- Il residuo a T=1 ($634/566 = 1.12$, quasi tutto nello scatter: 237 contro 180 ms) non è prefetch (entrambi i binari lo hanno, stessa distanza) e non è nemmeno pinning o NUMA: test dedicato (run_pinning.sh, job

697793), stesso nodo e stesso job, seq_opts sotto taskset -c 0 e sotto numactl --cpunodebind=0 --membind=0 dà tempi identici al run libero (621-625 contro 625 ms), mentre omp T=1 resta a 555. Restano come candidati le differenze di codegen (-fopenmp estrae il loop omp for in una funzione separata) e la struttura a cursori per-thread; causa non identificata, e va detta così. L'histogram identico alla cifra decimale garantisce che i binari sono confrontabili.

4. Deep dive: i costrutti OpenMP nel codice

La regione parallela di compute_phases (variante loop). Un'unica #pragma omp parallel copre histogram e scatter: si paga un fork/join solo, e le barriere interne sono quelle implicite di for e single. Sequenza: (1) single alloca i vettori di vettori; (2) ogni thread first-touch-a la PROPRIA riga di local_hists/cursors (località NUMA delle strutture di appoggio); (3) barriera esplicita; (4) for schedule(static) per l'histogram; (5) single per merge + prefix + cursori (vedi sez. 1); (6) for schedule(static) per lo scatter, stessi range del punto 4.

Attenzione al dettaglio dei timer (utile se lo chiedono): h_end è preso all'inizio del single del merge e s_start alla sua fine, quindi il costo del merge non appartiene né all'histogram né allo scatter nel breakdown. È comunque $O(T \cdot P + P) \approx 4000$ operazioni, microsecondi a $P=128$: invisibile. La frase del report "il costo del merge è in parte nascosto dalla regione successiva" è imprecisa: il merge non è nascosto (tutti i thread lo aspettano alla barriera del single), è semplicemente piccolo e non attribuito a una fase. Il vero gap con M2 sull'histogram (9 contro 35 ms a $T=8$) va raccontato come differenza fra i modelli di sincronizzazione attorno alla fase, non come merge nascosto.

reduction(+: join_count, checksum1, checksum2) nel join loop. Ogni thread accumula copie private e il runtime le combina alla fine: niente false sharing, niente atomiche nel loop. Nella variante task la stessa cosa è fatta a mano con PaddedResult thr_results[T] allineato a 64 byte (una cache line a slot, static_assert a garanzia), perché la reduction OpenMP non attraversa i confini dei task.

firstprivate nei task. pid, K, i range dei chunk: ogni task cattura il valore al momento della creazione. Necessario perché la variabile di loop cambia mentre i task restano in coda; shared sarebbe una race sul valore.

Il task annidato per le partizioni hot. Il task esterno costruisce tbl sul proprio stack, il taskgroup interno emette K sub-task di probe che la condividono in lettura e garantisce che il task esterno non termini (deallocando tbl) finché tutti i probe non hanno finito. $K = \min(T, \text{peso/media, con clamp a } 2)$: sub-task grandi circa quanto una partizione media, così entrano nella stessa economia di scheduling degli altri task.

5. Onestà: le sfumature dove conviene essere precisi

1. **Il nowait non è "required".** Misurato: nessuna differenza (la barriera implicita è un task scheduling point). Dire: "l'ho messo per non far attraversare ai worker una barriera inutile; la misura mostra che il runtime consuma i task anche senza".
2. **LPT conta poco qui.** Il vantaggio task sotto skew è dominato dallo split (15%) più che da LPT (2-5%). Il bound 4/3 di Graham resta la motivazione teorica corretta dell'ordinamento, ma con $P/T = 8$ la coda si autoalimenta comunque.
3. **La superlinearità non esiste.** Con baseline pari-ottimizzazioni l'efficienza a $T=4$ è 0.95. Il report lo dice già ("not genuine super-linear scaling"); ora c'è la misura.
4. **La distanza di prefetch del probe (8) non è ottima a $T=16$** (24-48 danno un altro 8% sul join, 5% sul totale, ~3 ms). E non era questione di tuning per-T: a $T=1$ le distanze 2-48 sono equivalenti, quindi una costante unica 16-24 era semplicemente migliore. Miglioria piccola ma reale, lasciata sul tavolo perché il valore fu calibrato a basso parallelismo.
5. **Il "merge nascosto" del confronto con M2** va riformulato: il merge è minuscolo (microsecondi) e nel breakdown M3 non è attribuito a nessuna fase (cade fra i due timer). Il gap sull'histogram con M2 viene dal modello di sincronizzazione, non dal costo del merge in sé.
6. **$P=128$ non è l'ottimo.** A $P=512-2048$ il totale scende del 20%+ (tabelle in cache): $P=128$ è tenuto per confrontabilità con il Modulo 2.

7. **Il tetto misurato (4.6) è leggermente sopra $1/f_{\text{hot}} = 4.4$:** f_{hot} pesa i record, non il tempo; il probe hot costa meno per record (tabella in L1). Il modello è un bound sul volume, la realtà lo supera di poco perché il costo unitario non è uniforme.
-

6. Cheat-sheet orale M3 (numeri da ricordare)

- Nodo: E5-2640 v2, 2x8 core, 32 HT, 2 NUMA. Input: NR=10M, NS=20M, P=128, max_key=5M.
- Report: seq 0.802 s; loop T=16 0.070 s (11.46x), task T=16 0.092 s (8.72x); M2 T=16 0.102 s (7.86x). A T=32 tutti a ~0.086-0.089 s (tetto di banda).
- Skew: $f_{\text{hot}} = 0.9/4 + 0.1/128 = 22.6\%$; hot/cold $\approx 290x$; peso hot = 29x la media; tetto join senza split $1/f_{\text{hot}} = 4.4$ (misurato 4.6), con split 5.5.
- Schedule: uniforme tutte uguali (chunk piccoli); skewed dynamic -38-40% vs static; dispatch dynamic sotto 1.3 ms anche a P=8192. Chunk 16 a T=32: solo 8 blocchi, degrada.
- Task: gap 31% a T=16 uniforme tutto in histogram+scatter (19+33 vs 6.6+17.4 ms); più task = gap minore, U poco profonda (minimo hist a 64, scatter a 512; oltre, ~1 μ s per task); divario residuo +4/+8 ms. nowait: zero misurato.
- Prefetch: totale 93 -> 62 ms a T=16 (1.49x); scatter 2.1x (37 -> 17 ms); probe 1.28x (49 -> 38 ms). Distanze: scatter U con plateau 8-16; probe piatto 2-48 a T=1 ma minimo a 24-48 a T=16 (-8%): costante unica 16-24 avrebbe dominato la scelta di 8.
- NUMA: first-touch sbagliato +13% totale a T=16 (+35% sullo scatter); interleave penalizza il join (+21%). A T=8 (un socket) nessun effetto.
- Baseline: seq 910 ms nuda, 634 con prefetch, omp T=1 566; eff a T=4: 1.36 -> 0.95.
- Validazione: triple identiche fra ogni variante/schedule/ordine degli esperimenti (uniforme: join_count 40006682); naive $O(N^2)$ sui piccoli; skewed vs parallelo T=1.